

Technical Document

For AI Enabled Regression Test Optimization

Using AI to optimise regression testing, automated with pytest

Version 1.3 | 25 September 2026

Table of Contents

Part 1 - The Framework.....	5
1. Objective.....	5
2. Design Principles.....	5
3. System Architecture.....	6
4. Where the AI Is Used	7
4.1 The decision.....	7
4.2 What the model is given	7
4.3 What the model returns.....	8
4.4 Guardrails	8
4.5 Model engines.....	8
4.6 The rule planner: baseline and fallback	9
4.7 Current limits and next steps	9
5. Change Analysis	10
6. Risk Model	10
7. Planning.....	11
7.1 Rule planner.....	11
7.2 Scenario vocabulary.....	11
7.3 Worked examples	12
8. Test Generation with pytest.....	13
8.1 The planner-selected module.....	13
8.2 Requirement and release-note module.....	13
9. Governance and Framework Instrumentation.....	15
9.1 Acceptance rules and approval	15
9.2 KPIs	16
9.3 Coverage and trend	16
10. Execution, Reporting and Escalation.....	16
11. Porting to Another Target.....	17
Part 2 - Proof of Concept: nRF52840	18
12. Why This Platform	18
13. Transport over GATT	18
13.1 Device resolution.....	18
13.2 Stream segmentation and termination.....	19
13.3 Latency and loopback probe	19
13.4 Per-metric degradation.....	19
14. Target Firmware	19

14.1 Deferred notification.....	19
14.2 Advertising restart	19
14.3 Identity address.....	19
15. WDRC Compressor.....	20
16. Telemetry Wire Format	20
17. Bench Measurements and Results	21
18. Supporting Subsystems of the Adapter.....	22
18.1 Audio conversion.....	22
18.2 Path resolution	22
18.3 Optional UART console capture.....	22
18.4 Loopback failure analysis	22
Part 3 - Build Procedure	23
19. Prerequisites.....	23
20. Host Package	23
21. Target Firmware (PoC).....	23
22. Executable Build.....	24
23. Verification and Distribution Readiness.....	24
Part 4 - Reference	25
24. Licensing and Third-Party Components	25
25. Source File Reference.....	25
25.1 Generic core.....	25
25.2 Proof-of-concept adapter and assets	26
25.3 Tests and tools	26
26. Project Structure	27

AI Enabled Regression Test Optimization - Technical Document

Version: 1.3

Scope: Architecture of an AI-enabled regression test suite; the AI planner and its guardrails; test generation with pytest; porting to a new target; the nRF52840 proof of concept; the build procedure.

Audience: Engineers adopting, extending, porting or reviewing the framework.

Licence: MIT. Copyright (c) 2026 Prajnaana Technologies Pvt. Ltd. Original author: Dhanya Shree S.

Operation of the packaged application is described in the accompanying User Guide.

Part	Contents
Part 1 - The Framework	Objective, architecture, where the AI is used, change analysis, risk, planning, pytest generation, governance, KPIs, porting
Part 2 - Proof of Concept: nRF52840	Why the platform was chosen, and the adapter, firmware, transport and measurements that implement it
Part 3 - Build Procedure	Building, programming, packaging and verifying
Part 4 - Reference	Licensing, source files and project structure

Part 1 - The Framework

1. Objective

This project is an open-source reference for **using AI to optimise regression testing, and for automating the optimised suite end to end with pytest**, on real hardware.

Regression testing on every build is the safe policy and the expensive one. On embedded products the expense is bench time: a device under test serves one connection, tests run in real time, and a full suite can take longer than the interval between builds. The usual response is that an engineer chooses a subset by judgement. That is fast, but it is not repeatable, not recorded, and only as good as the engineer's knowledge of the change.

The framework replaces that judgement with an explicit pipeline:

1. **Evidence.** Work out what changed, from the release note, the built firmware image or the source tree, and read the state of the device.
2. **Decision.** An AI planner chooses the smallest set of tests, and the stress intensity, that would catch a regression in that change. A rule-based planner is the default and the fallback.
3. **Automation.** The chosen tests are generated as a pytest module, checked mechanically, run against the device, and reported in JUnit XML. Requirements and release notes are turned into pytest cases automatically on every run.
4. **Feedback.** On failure the intensity is raised and the run repeated; every run is recorded so the framework can measure whether it is working.

To show that this works outside a simulator, the framework is applied to a **proof-of-concept target**: a Nordic nRF52840 development board running Zephyr firmware whose audio compressor stands in for a hearing aid's signal processing, reached over Bluetooth Low Energy. The board demonstrates the approach; it is not its subject. Everything specific to it lives in a replaceable adapter layer (section 3) and is described in Part 2.

2. Design Principles

Principle	What it means in the design
The AI decides, templates execute	The model chooses from a closed vocabulary of scenarios and intensities. It never writes executable code and never talks to the device. A hallucinated answer can therefore select a wrong test, but never an unsafe or non-existent one.
Evidence before judgement	The change is derived from artefacts - the release note, the firmware image, the source tree - rather than typed by a person. Typing the change so the planner can key off it would be choosing the tests by hand.
Offline by default, graceful fallback	A fresh checkout and a CI runner work with no credential and no network. The AI planner is opt-in, and any failure in it degrades to the rules with a diagnostic.
Deterministic where it must be	Parsing requirements and release notes is rule-based, so the same documents always produce the same suite unless the operator selects With API (HA_AI=llm), which lets a model write the requirement cases (section 8.2). The determinism KPI depends on it.
Unmeasured is not healthy	A metric the device does not report scores as risk, and a test that cannot be judged is skipped with its reason - never passed.
Generic core, replaceable adapter	The core never names the device. Transport, telemetry, measurements and scenario templates are the adapter, and are the only code a new target replaces.

3. System Architecture

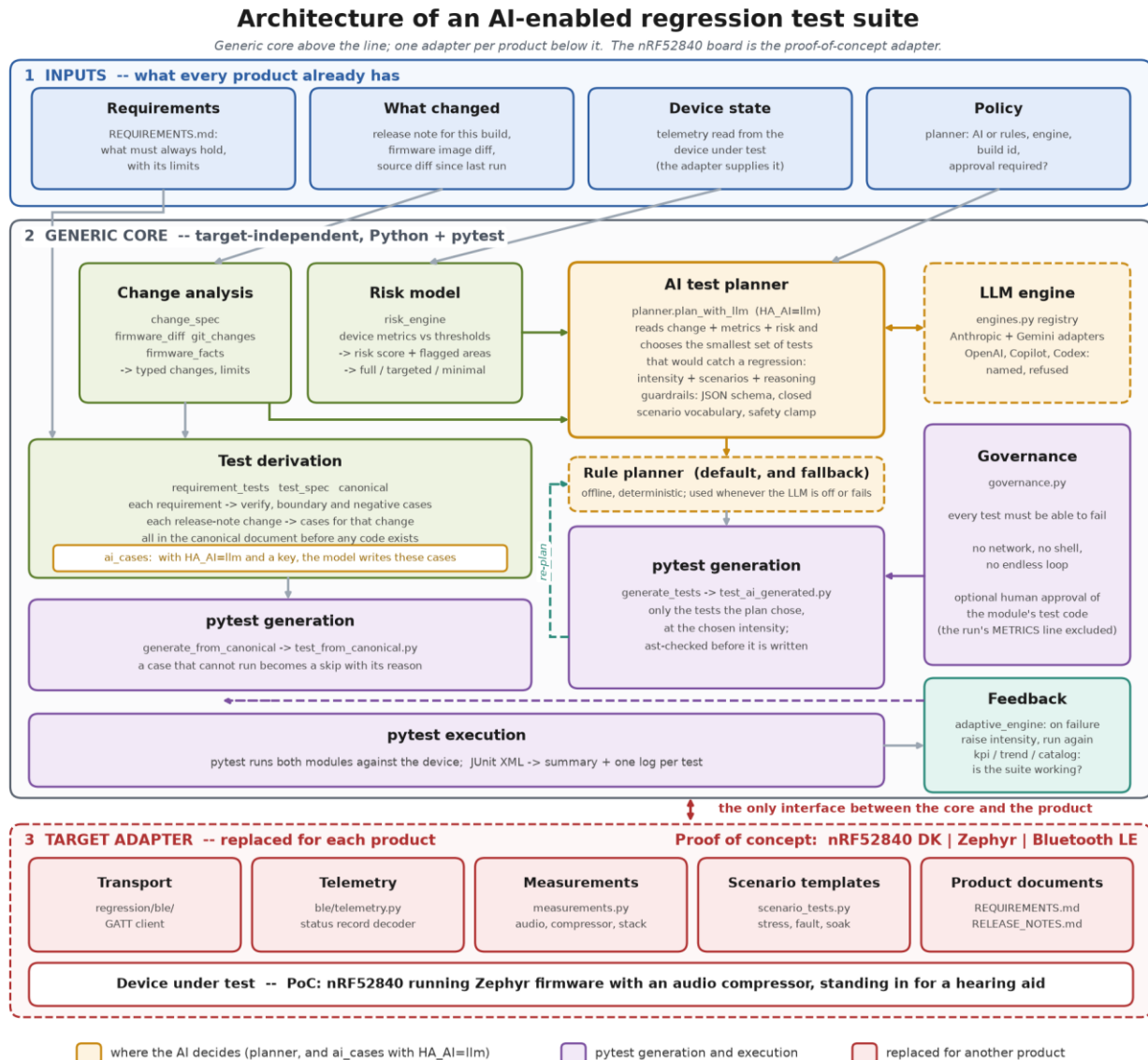


Figure 1 - Architecture of an AI-enabled regression test suite.

The architecture has three layers. Inputs are what any product already has. The generic core is target-independent Python and pytest. The target adapter is replaced for each product; the proof-of-concept adapter drives the nRF52840 over Bluetooth LE.

Layer	Component	Module	Responsibility
Core	Orchestration	ai_engine/orchestrator.py	Sequences the pipeline; owns escalation and KPI recording.
Core	Change analysis	change_detection/firmware_diff.py, git_changes.py, firmware_facts.py; ai_engine/change_spec.py	Works out what changed and turns release-note prose into typed changes.
Core	Risk model	change_detection/risk_engine.py	Metrics against thresholds, a score, and a regression slice.
Core	AI planner	ai_engine/planner.py, engines.py, llm_client.py, gemini_client.py	Chooses intensity and scenarios; rules as default and fallback.
Core	Test derivation	ai_engine/requirement_tests.py, test_spec.py, ai_cases.py, canonical.py	Requirements and changes to test cases, collected in one canonical document

			(built in memory on every run; python -m regression.ai_engine.canonical --out FILE writes it for inspection).
Core	pytest generation	ai_engine/generate_tests.py, generate_from_canonical.py	Emits the two pytest modules.
Core	Governance	governance.py	Acceptance rules and approval for generated suites.
Core	Execution and reporting	reporting.py; pytest	Runs the modules, JUnit XML to per-test logs.
Core	Feedback	ai_engine/adaptive_engine.py, kpi.py, analytics/trend.py, catalog.py	Escalation, KPIs, trend, coverage by category.
Core	Presentation	dashboard.py	Tkinter front end over the same pipeline.
Adapter	Transport	ble/ble_audio.py	PoC: GATT client, streaming, metric reads.
Adapter	Telemetry	ble/telemetry.py	PoC: decoder for the device status record.
Adapter	Measurements	measurements.py, audio_prep.py, instrumentation/power_analyzer.py	PoC: audio and compressor measurements, stimulus conversion, external current.
Adapter	Scenario templates	ai_engine/scenario_tests.py	PoC: the test code each scenario emits.
Adapter	Product documents	REQUIREMENTS.md, NRF_Firmware/RELEASE_NOTES.md	PoC requirements and release note.

4. Where the AI Is Used

4.1 The decision

With HA_AI=llm the AI makes two decisions. The first is which tests are worth running given this change and this device state, and how hard (the planner). The second is the requirement and release-note cases themselves (HA_CASE_SOURCE, section 8.2). The first is where test optimisation happens: it is a judgement over heterogeneous, partly free-form evidence - a release-note sentence, a set of metrics some of which are missing, a risk score - which is what a language model does well and a threshold table does poorly.

Stage	Decided by	Reason
Test selection and intensity	AI planner (HA_AI=llm)	Judgement over free-form evidence
Test selection, default and fallback	Rule planner	No key, no network, no cost; always available
Risk score	Threshold table	Every score traceable to a specific comparison
Reading requirements and release notes	Deterministic parsers; a model writes the cases only under HA_AI=llm	Same input, same suite
Writing test code	Templates	The model never writes executable code
Executing tests	pytest	-

4.2 What the model is given

planner.build_prompt assembles a single request:

- The device metrics, as JSON, with any metric the device did not report listed under "unmeasured" and the instruction to treat it as unknown, never as good.
- The change under test, as derived in section 5.
- The risk engine's output: the score and the flagged test areas. On a full regression every area is listed as flagged ("nothing was excluded"), the same reading the rule planner uses.
- The last five runs (kpi.planner_history): build id, verdict, intensity, scenarios and the names of the tests that failed. Timings are left out to keep the prompt small and focused.

- The system prompt (SYSTEM_PROMPT), which describes the target and its metrics, quotes the risk engine's limits (read from METRIC_THRESHOLDS, so the model and the rules apply the same numbers), and tells the model to prefer the lowest intensity the evidence allows, because every step up costs bench time.

4.3 What the model returns

The request is made with a JSON-schema output constraint (PLAN_SCHEMA), which the Gemini client translates into the OpenAPI subset that service accepts:

```
{ "intensity": "low" | "medium" | "high",
  "scenarios": [ zero or more of KNOWN_SCENARIOS ],
  "reasoning": "why these were chosen" }
additionalProperties: false
```

The reasoning is printed with the plan and shown in the dashboard, so every AI decision is recorded with its justification.

4.4 Guardrails

Guardrail	Mechanism	Effect
Schema constraint	PLAN_SCHEMA declares scenarios as an enum over KNOWN_SCENARIOS, passed as the structured-output format	A scenario the generator cannot emit is not expressible on either engine while the schema is sent; on Gemini's retry without a schema, the post-hoc clamp removes it.
Post-hoc clamp	RegressionPlan.__post_init__ intersects the scenarios with KNOWN_SCENARIOS and clamps packets to MAX_PACKETS (80)	A plan can never overload the device
Vocabulary contract	A host test holds KNOWN_SCENARIOS and the generator's templates in step	Every selectable scenario produces a real test
Exception containment	Missing package, missing credential, transport error, refusal, and JSON or schema violations are caught	The run falls back to the rules and says why
Governance	Every generated module is checked before it runs (section 9)	A test that cannot fail is reported, and can block the run
Data minimisation	Planner: metrics, the change description, the risk output and a summary of recent runs. Case generation (HA_AI=llm): the requirement document and the release-note section for this build	No firmware and no audio leave the machine, and no credential besides the API key itself
Closed case vocabulary	A model proposing requirement cases may use only the ten actions and nine measurements declared in ai_cases; they are named in the request schema and re-checked on the reply	A step outside the vocabulary makes a named skip, never an executed case; a proposal with an unknown category or priority, or with no description, is discarded

4.5 Model engines

regression/ai_engine/engines.py defines the Engine interface (available, complete_json, model_name), a registry, and a selection policy (HA_ENGINE). Two adapters are implemented: anthropic, the default, through llm_client.py with default model claude-opus-5 and effort low (HA_LLM_MODEL, HA_LLM_EFFORT); and gemini, through gemini_client.py, default model gemini-3.6-flash (HA_GEMINI_MODEL). Each adapter is a thin wrapper whose client module owns the credential, the request and the schema translation, so a second engine is a thin object and not a second pipeline. The Gemini client uses only the standard library (urllib), so it adds no dependency; it reads GEMINI_API_KEY, or GOOGLE_API_KEY when that is unset, retries a 503 up to three attempts, and retries a request the service rejects with a 400 once more without the response schema, restating the schema in the prompt. OpenAI, Copilot and Codex are named and refused with a clear "not implemented" error rather than registered as stubs. Adding an engine is three steps in the core: implement the interface, register it, and remove it from NOT_IMPLEMENTED. The window keeps one table of its own, the key prefixes it recognises, so an engine whose keys should be checked as they are pasted adds a fourth edit there.

A person types the key into the application window, or sets it in the environment before starting. CI needs no key: every job in regression.yml runs HA_AI=rules. A CI job that wanted the model would take the key from a repository secret exposed as an environment variable. Each engine declares the variable it reads (key_env), so the dashboard's key box asks the engine rather than keeping its own copy of the mapping. The key is never saved. It is held in the running process, passed to the runs that process starts, and gone when the window closes: there is no credential store, no file to clear and nothing left behind. That is the reason for the design - a key that is never written down cannot be found later on a shared machine, cannot travel inside a zipped folder, and cannot be committed. Every other setting the framework reads is an environment variable too, including GEMINI_API_BASE, ANTHROPIC_AUTH_TOKEN and HA_LLM_FALLBACKS. They are not repeated here: README.md's "Every variable" table is the single list, with each default and which of them the application window sets for you.

Verification against a live model. The Gemini path has been exercised end to end against a live key on the free tier. Given a degraded metric set and the change "bluetooth telemetry, pairing security", the model returned intensity high and five scenarios from the closed vocabulary - concurrency_stream_and_read, memory_pressure, pairing_bonding, power_stress and reconnect_stress - and its stated reason cited the breached limits (memory 3.9 KiB against 3.76 KiB, power 72 mA against 60 mA) and the risk score; every name was valid, so the post-hoc clamp removed nothing. The same key exercised ai_cases, which proposed requirement cases the rules do not derive, each carrying its one-sentence justification and each re-validated against the closed action and measurement vocabulary before it was kept. The failure paths were exercised rather than simulated: a retired model alias returned 404 naming its replacement, the free tier returned 429 once its daily quota was spent, and the service returned 503 under load. In each case the run continued on the rule-based path and printed the reason; no run failed because a model could not be reached. Two limits stand. Case generation over the whole requirement document has not completed against a live model: the Gemini free tier refused a request of that size, so only a request scoped to one section has completed. The Anthropic adapter is covered by the offline tests; its live path has not yet been exercised against a live key.

4.6 The rule planner: baseline and fallback

The rule planner receives the same inputs and returns the same RegressionPlan type, so the two planners can be compared run for run on the same build. Its rules are in section 7.

4.7 Current limits and next steps

Item	State
Reduction scope	test_reduction (section 9.2) measures the planner-selected module; the requirement and release-note module always runs in full.
Model-backed document reading	change_spec and canonical are deterministic unless HA_AI=llm, when a model may write the requirement cases (section 8.2). A model-backed reader for prose the rules cannot parse would produce the same canonical document, and would be confined to it; none is written.
Other engines	Anthropic and Gemini have adapters. OpenAI, Copilot and Codex are named in the design and refused by name.
Live-model coverage	The Gemini adapter has been exercised against a live key, including its failure paths (section 4.5). The Anthropic adapter is covered by the offline tests; its live path has not yet been exercised against a live key.
Model-proposed cases at document scale	Not verified over the whole requirement document against a live model. Only a request scoped to one section has completed; the Gemini free tier refused a request of that size, so the limit is the provider's quota rather than the mechanism.
Model-written test bodies	No model-written code is executed. A case a model proposes is data: Python renders it through the same fixed templates as any other case, and governance checks the rendered module before it runs.

5. Change Analysis

orchestrator.derive_change works out what changed, preferring evidence over description:

Order	Source	Module	What it gives
1	Release note section for the build on the device	change_spec.py	What the team says changed. When given, it is the only thing selection reads.
2	Built firmware image against the last tested build	firmware_diff.py	Every symbol and its size, compared: what actually changed on the device, even on a dirty build. The image's SHA-256 is stored with it: when no symbol moved but the bytes differ, the comparison gives no description, so the source tree (row 3) is consulted.
3	Source tree since the last tested revision	git_changes.py	Files changed, mapped to subsystems.
4	Nothing	-	The text "unknown change". It adds nothing to the risk score (which reads metrics only) and triggers no scenario; the metrics alone decide the plan.

The firmware fingerprint is filed on every run, whether or not it is the source selection uses, so the next run always has something to compare against.

--change "text" on the command line overrides all four (manual use only).

Release notes to typed changes. change_spec reads each bullet into a type (new_feature, enhancement, bug_fix, configuration_change, removal, documentation), a component, a description and any stated values, for example {"from": 64, "to": 128, "unit": "bytes"}. A bullet is read whole, including wrapped lines; its component comes from the bullet or its #### heading. The bullet is then split on "and" and on commas, and each clause is read on its own: a clause whose object is a comment, a typo, a licence header or the docs is typed documentation and derives no case, while a real change in the same sentence keeps its cases.

Claims against the image. firmware_facts checks each claimed change against the build's .config, symbols, strings and memory map, and returns CONFIRMED, CONTRADICTED or UNVERIFIABLE. Only CONTRADICTED is a warning; most sentences in a release note leave no trace in an artefact, and treating that as failure would train everyone to ignore the warnings.

6. Risk Model

change_detection/risk_engine.py compares each metric with a threshold. A breach, or a metric the device did not report, adds the metric's weight to the score and flags a set of test areas. The values below are the proof of concept's; the metrics, thresholds and weights are the part of the risk model a new target replaces.

Metric	Unit (PoC)	Threshold	Weight	Flags	Calibration
power	mA supply current	60	2	battery, power_stability	Uncalibrated; needs an external analyser. The figure is REQ-PWR-001's
memory	KiB of work-queue stack high-water	3.76	2	memory_leak, stability	Calibrated from 24 samples of an earlier build; awaits recalibration
sync	ms binaural sync error	50	1	connection, sync	Uncalibrated; needs two devices. The figure is REQ-SYN-001's
retry	unexpected link drops in the run	5	1	retry, reconnect	First guess; healthy is 0

The memory limit sits halfway into the headroom between the observed plateau (3.53 KiB) and the 4.00 KiB stack, so it fires before exhaustion and cannot fire on a healthy device. That calibration predates ef509275, which

makes the metric sample the work queue; it now reads about 0.52 KiB, so the limit awaits recalibration. Power and memory carry double weight because they degrade monotonically over a deployment; sync and retry are more sensitive to the bench's radio environment.

Score	Slice	prioritized_tests
4 and above	Full	None - nothing is narrowed; both planners treat every flagged area as flagged
2 to 3	Targeted	The flagged test areas
0 to 1	Minimal	["connection"]

Unmeasured metrics are reported separately from breaches, so the console says whether a run escalated because the device misbehaved or because it could not be read. HA_REQUIRE_TELEMETRY=1 turns an unmeasured memory or retry reading into an error; power and sync may stay unmeasured.

7. Planning

plan_regression dispatches on HA_AI and returns a RegressionPlan: intensity, scenarios, reasoning, source (rules or llm) and the packet count.

The intensity is written into the generated module (INTENSITY, PACKETS) and sets how hard the stress scenarios push:

Intensity	Packets	Streams (memory / power stress)	Reconnect cycles
low	20	5 / 3	2
medium	50	12 / 6	3
high	80 (MAX PACKETS)	20 / 10	5

7.1 Rule planner

The rule planner applies the risk engine's limits (METRIC_THRESHOLDS). decide_intensity: high if the change mentions Bluetooth or power is over its limit; medium if memory is over its limit, or if power or memory is unmeasured; otherwise low. An unmeasured metric raises the intensity one step rather than to the top, because the risk engine has already scored it as risk and widened the slice. suggest_scenarios adds scenarios from the metrics, from the flagged areas (memory_leak, stability, connection, retry, sync), from a risk score of 3 or more (a full regression counts every area as flagged), and from words in the change description (CHANGE_TRIGGERS, matched at word boundaries so that "during" does not read as "ring").

On failure the orchestrator re-plans with the escalated intensity as a floor (generate_tests min_intensity), so the re-run is always harder than the run that failed, and the escalated plan is the one recorded for the KPIs.

7.2 Scenario vocabulary

Scenario	Category	What it catches
reconnect_stress	fault_injection	Recovery code that has never actually run
fault_link_drop	fault_injection	A link lost mid-transfer, and whether the stream recovers
stress_stability	stress	Where the throughput limit is, and whether crossing it is graceful
memory_pressure	stress	Memory that climbs across repeated streams and never returns
power_stress	stress	Current draw under sustained streaming
long_duration_stability	long_duration	Leaks, counter overflow and drift (HA LONG MINUTES)
concurrency_stream_and_read	concurrency	Telemetry read during streaming
pairing_bonding	security	Whether encryption is enforced, and whether a bond survives reconnection

7.3 Worked examples

Produced by the rule planner from the code as delivered, with metrics supplied on the command line.

Inputs	Risk	Plan
power 80, memory 3.9, sync 10, retry 0; change "bluetooth reconnect fix"	power and memory breach: score 4, full regression	high (80 packets); seven scenarios: power_stress and memory_pressure (breaches); long_duration_stability, stress_stability, reconnect_stress, fault_link_drop, concurrency_stream_and_read (a full regression flags every area). Only pairing_bonding is left out. 18 of the 21 planner tests are selected.
power 40, memory 3.1, sync 10, retry 0; change "readme update"	no breach: score 0, minimal (connection)	low (20 packets); fault_link_drop, concurrency_stream_and_read, because the minimal slice flags the connection. 13 of the 21 planner tests are selected.
PoC bench as measured: memory 0.52, retry 0, power and sync unmeasured; change "Removed the knee from the WDRC compressor"	power and sync unmeasured: score 3, targeted (battery, connection, power_stability, sync)	medium (power unmeasured raises one step); power_stress, fault_link_drop, concurrency_stream_and_read, reconnect_stress. 15 of the 21 planner tests are selected.

8. Test Generation with pytest

How a run is put together

the release-note section read here is the newest one

every count below is rules mode, what a clean clone of this repository produces: no firmware build tree, so no .config is read

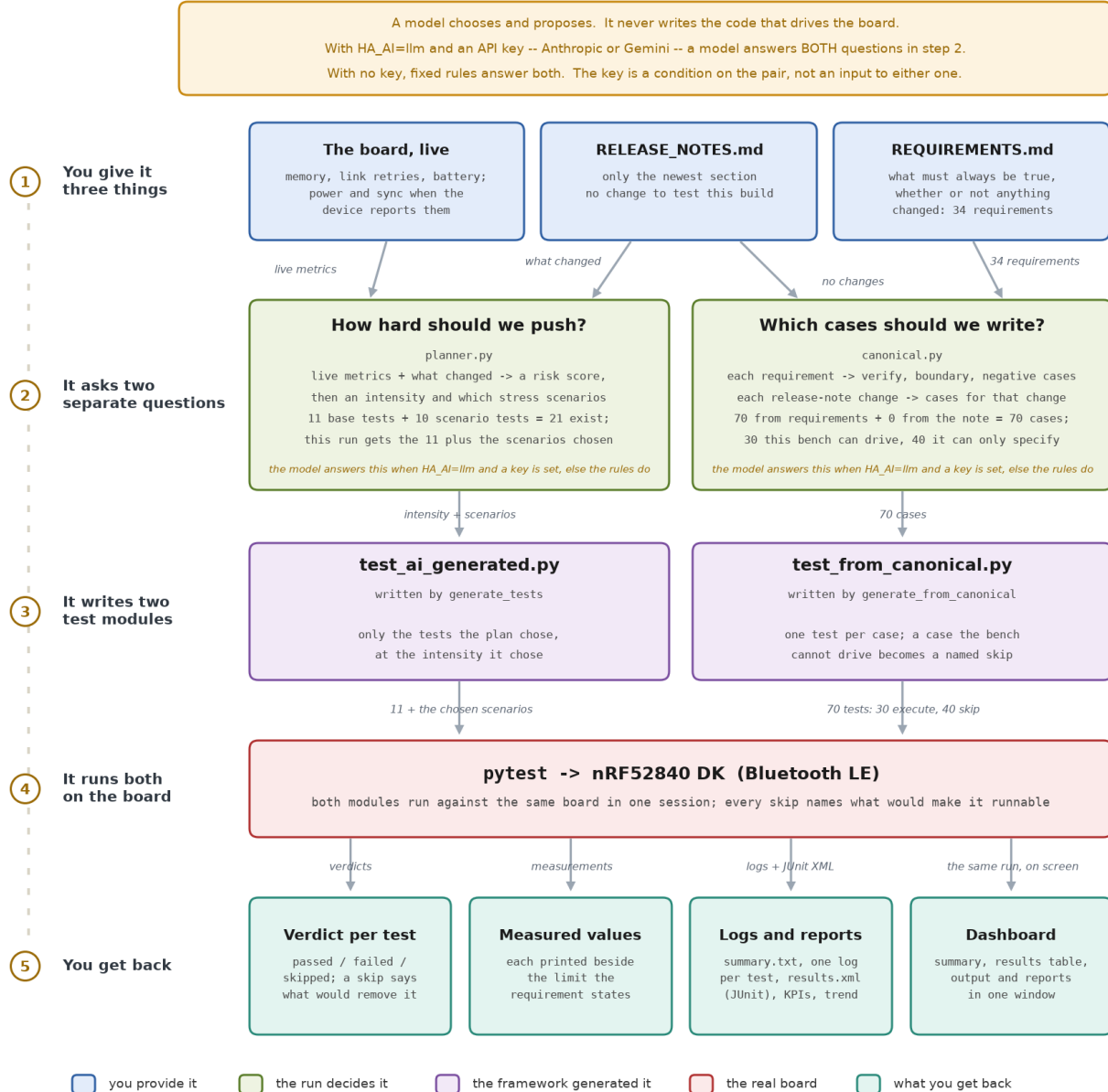


Figure 2 - How a run is put together: inputs, the two questions, the generated modules, the run and the outputs.

8.1 The planner-selected module

generate_tests composes test_ai_generated.py as source text, validates it with ast.parse, and writes it. The module always contains the connection test, the target's baseline tests and one bounds test per metric, and appends the template of each scenario in the plan. Metrics, thresholds and measurement constants are written in as literals, so the file records exactly what the run executed. The session fixture in conftest.py hands every test one connected device.

8.2 Requirement and release-note module

Every run also writes test_from_canonical.py, through one intermediate document:

```

REQUIREMENTS.md --> requirement_tests.py --+
                                     +--> canonical document
RELEASE_NOTES.md --> change_spec.py -----+   (canonical.py)
                                     |
                                     v
                                     test_from_canonical.py
                                     (generate_from_canonical.py)

```

Stage	Module	Output
Release note	change_spec.py, test_spec.py	Typed changes, then the cases each change type implies: a configuration change stating a limit gives cases at and past it; a memory fix gives repetition.
Requirements	requirement_tests.py	A Verify case per requirement, plus boundary and negative cases at each limit it states. Limits can also come from the build (for example CONFIG BT MAX CONN).
Model-proposed cases (optional)	ai_engine/ai_cases.py	A third source: cases a model proposes, each with a one-sentence justification, stamped source "llm" and numbered AI-001 upward. Every mode of it needs HA_AI=llm - the switch the window sets from With API - so a run asked to stay offline reaches no model, whatever key the machine holds. HA_CASE_SOURCE then chooses among three behaviours: auto (the default) and ai have the model write all the cases, requirement and release-note alike, and the rule-derived cases are used only if it returns none; both keeps the rule cases and adds up to HA_AI_CASE_LIMIT (12) model extras - prose stating no number, interactions between requirements, implied failure modes; rules never calls a model. Any failure - no key, a rate limit, malformed JSON - leaves the rule-derived cases standing and prints why. A model measure step that states no minimum or maximum makes its case not runnable.
Merge	canonical.py	The canonical document (in memory): per case, setup, steps, expected conditions, timeout, cleanup, an executable flag and a skip reason.
Emit	generate_from_canonical.py	test_from_canonical.py. A case that cannot run is emitted as pytest.skip with its reason, never dropped.

The canonical document (in memory) is the seam between analysis and execution: everything upstream reads prose, everything downstream only follows steps, and nothing re-reads a sentence when it emits a test.

Requirement format. One per line: an identifier (REQ-, SRS-, FR- or NFR-), a colon and one sentence. The identifier's second segment names the component. Limit words: under, below, less than and fewer than give an exclusive upper limit; at most, not exceed, no more than, within, up to and shall not be above an inclusive one; above, more than and greater than an exclusive lower limit; at least and no less than an inclusive one; between X and Y an inclusive range. A negated bound -- "not" or "never" before more than, above, below or less than -- states the opposite side, inclusively: "shall not grow by more than 0.25 KiB" is a ceiling of 0.25, and "must never go below 3 dB" a floor of 3. No greater than and no fewer than work like no more than and no less than.

Measured requirements. canonical.MEASURED maps requirement wording to a measurement the adapter provides; the limit is always read from the requirement, so editing the requirement moves the limit. On the proof-of-concept board, removing REQ-AUD-010 removed its three cases; adding "latency shall be under 500 ms" added a case that passed; adding "stack high-water shall not exceed 0.25 KiB" added one that failed against the 0.52 KiB the current build reads. The verdict follows the requirement, not the code.

As delivered, the proof-of-concept requirement document's 34 requirement lines derive 70 cases (rules mode) from a clone of this repository, of which 30 are runnable on this bench and 40 are specified but not runnable. The current build changes only comments, so its release-note section adds no cases at all. On a machine that also has a

firmware build directory the count is 73 rather than 70 (rules mode again): the build's .config supplies CONFIG_BT_MAX_CONN, which parameterises three extra concurrent-connection cases. The figures in this document quote the clone numbers, because a reader can reproduce those. The Noise cancellation requirements (REQ-ANC-001 to REQ-ANC-003) are stated although the proof-of-concept firmware does not implement the feature, on the principle that the suite's job is to say what is untested rather than to stay silent about it. The identifier is what keeps them honest: ANC maps to a component the bench cannot drive, so every case derived from those three is refused before it runs. Without that mapping the classifier falls back to the prose, and REQ-ANC-003 -- which mentions the compressor -- reads as an audio case that runs and passes against firmware with no noise cancellation at all.

How many tests? With the default rule-based planner, the same requirements always produce the same tests: for the documents in this repository, 70 cases, of which 30 run on the proof-of-concept board. With an AI model (HA_AI=llm) the model writes the cases, so the number varies from run to run. Each run prints its own count.

9. Governance and Framework Instrumentation

9.1 Acceptance rules and approval

A generated test that asserts nothing passes forever and is counted as coverage. governance.py checks every generated module against mechanical rules:

Rule	Requirement
parses	The module is valid Python
has_tests	The module defines at least one test
must_assert	Every test asserts or skips, so it can fail
unique_names	No two tests share a name (pytest would silently run only the last)
no_network	No test imports a network or process module (FORBIDDEN_IMPORTS: subprocess, shutil, ctypes, socket, requests, urllib, urllib3, http, ftplib, telnetlib, smtplib). This rule reads import statements; reaching one of those modules without importing it, as an attribute of a package that did, is refused by no_escape.
no_escape	No test calls, imports, or reaches through getattr a forbidden function (FORBIDDEN_CALLS: shell and process start, eval/exec/compile/_import_, file deletion and renaming, and globals/vars/locals), and no test reads one of those namespace builtins even without calling it. No test uses anything from os, asyncio, time or wave beyond the short list the templates need (GUARDED_MODULES). No test reads a dunder name or a dunder attribute, and no test calls the result of a subscript, because both put the callee in a string the test builds rather than in a name a rule can read. An attribute chain is read whole, not two deep: a chain naming os, sys, ctypes, ctypeslib, builtins, importlib or any FORBIDDEN_IMPORTS module at any depth is refused — np.sys, np.ctypeslib.ctypes and regression.paths.os are each that module reached through a package the import allowlist permits — and an attribute name built out of one of those, such as subprocess_exec on an event loop, is refused whatever object it is taken from. A chain through np or the regression package may reach only the helpers the templates emit (PACKAGE_SURFACE).
must_terminate	No test loops forever
no_top_level_code	Nothing runs when the module is imported: only its docstring, imports, definitions and constants assigned to plain names. It may import only pytest, numpy as np, os, time, asyncio, wave and regression, each under that one name (ALLOWED_IMPORTS); the only decorator is @pytest.mark.*; and no module-level name may rebind a name the rules decide by (PROTECTED_NAMES), define a dunder such as __getattr__, or pass a class keyword. The same allowlist applies inside a class body, which runs at import exactly as the statements around it do.

Approval is a recorded human decision against the module's test code: every byte except the run's measured METRICS line, and that line is set aside only when its value is a Python literal, so code cannot be hidden behind

the name. Editing any test invalidates it. HA_REQUIRE_APPROVAL=1 refuses to run an unreviewed or rule-breaking suite.

python -m regression.governance validate | approve | status act on both generated modules (--path FILE, before or after the command, narrows to one); approve requires --by NAME, so every approval carries the name of the person who gave it. Approve after a run: an approval covers the module's test code, not the measurements of the run that produced it. Under HA_REQUIRE_APPROVAL=1 an escalated re-run generates new, unapproved bytes, so the escalation is refused, and the first run's result stands and is recorded.

9.2 KPIs

Each KPI reports a value only when the data supports one; otherwise it reports INSUFFICIENT DATA and what it needs.

KPI	Question	Target
determinism	Run the same build twice - same result?	98 %
early_detection	What share of defects was caught before integration?	70 %
cycle_predictability	Does a regression run take a consistent time?	10 % deviation or less
escape_rate	How many defects were found after sign-off?	30 % reduction
maintenance_effort	Does keeping the suite alive grow with feature count?	No growth (person-days per release)
test_reduction	How much of the full suite did the planner leave out, and how much bench time did that save?	Tracked, no target
subjective_stability	Is perceptual rework increasing?	Trend only

test_reduction is the measure of the optimisation itself. Each run records how many tests the plan generated against the full suite - the same module with every scenario in the vocabulary - and which tests it left out. Bench time saved is estimated from how long each left-out test took in runs that included it; a test that has never run is reported as untimed, not guessed.

Determinism is established first: if two runs of one build disagree, every other measure is computed over unreliable verdicts. A failing run records the defects it caught, which feeds early_detection without manual entry.

9.3 Coverage and trend

catalog.py tags tests as functional, stress, fault_injection, long_duration, concurrency or security, discovering automated tests by parsing the suite and declaring manual and subjective work by hand. It measures the 65/25/10 automated / targeted-manual / subjective split and names any category with no tests. analytics/trend.py compares runs, flags newly failing tests, detects slow creep across builds, and names tests that gave different verdicts on the same build.

10. Execution, Reporting and Escalation

pytest is invoked with --junitxml and -o junit_logging=all. reporting.parse_junit produces one record per test; write_test_logs writes one log file per test; emit_results prints a marker line per test that the dashboard parses without reading the XML. Each run writes regression/logs/build_<timestamp>/ with summary.txt, results.xml and tests/<name>.log.

adaptive_escalation advances low to medium to high on any non-pass. The orchestrator re-plans with that intensity as a floor and runs once more (refused under HA_REQUIRE_APPROVAL=1; see 9.1). At high it reports that nothing further exists. The final plan, with its selection counts, is then recorded for the KPIs under its build id.

11. Porting to Another Target

The generic core stays as it is. A new product supplies the adapter:

Replace	PoC implementation	Contract
Transport	ble/ble_audio.py (RealBLEDevice)	Find and connect to the device; send stimulus and collect the response; read metrics. Any link: UART, USB, Ethernet, CAN, a debug probe.
Telemetry	ble/telemetry.py	Return a dict of metrics, listing anything not measured under "unmeasured" - never a substituted zero.
Measurements	measurements.py	Return numbers, never verdicts, for each quantity the requirements state a limit for.
Scenario templates	ai_engine/scenario_tests.py	The test source each scenario emits; keep KNOWN SCENARIOS in step.
Fixture	confest.py, regression/generated_tests/confest.py	A session fixture that yields a connected device.
Product documents	REQUIREMENTS.md, RELEASE_NOTES.md	The product's own, in the same format.

And retunes eight tables in the core:

Table	File
Metrics, thresholds, weights, flagged areas, UNMEASURED WHY	change_detection/risk_engine.py
Component vocabulary for release notes	ai_engine/change_spec.py
Requirement wording to measurement (MEASURED), identifier components	ai_engine/canonical.py
The system prompt describing the target to the model	ai_engine/planner.py (SYSTEM_PROMPT)
Symbol to subsystem	change_detection/firmware_diff.py (SYMBOL_SUBSYSTEMS)
Path to subsystem	change_detection/git_changes.py (PATH_SUBSYSTEMS)
Change words to scenario	ai_engine/planner.py (CHANGE_TRIGGERS)
Case-generation prompts and vocabulary	ai_engine/ai_cases.py (SYSTEM_PROMPT, SOLE_SYSTEM_PROMPT, ACTIONS, MEASUREMENTS)

Then calibrate the thresholds from a known-good build (tools/capture_baseline.py), run the host-only suite, and register a self-hosted CI runner with the device attached.

Part 2 - Proof of Concept: nRF52840

12. Why This Platform

An optimiser is only worth demonstrating where regression testing is expensive. The proof of concept was chosen for three properties:

Property	Consequence for testing
Hardware in the loop	The behaviour under test is DSP executing on a Cortex-M4F. Tests run against the physical board over GATT; there is no simulator.
Serialised bench	CONFIG_BT_MAX_CONN is 1 and one board is available, so every test costs real bench time and selection matters.
A real function to regress	A wide dynamic range compressor, the core function of a hearing aid, gives the tests a measurable behaviour to protect.

The board is a Nordic nRF52840 DK (PCA10056): single-core Arm Cortex-M4F with FPU, 1 MB flash, 256 KB RAM and a built-in Bluetooth LE radio. The firmware advertises as Zephyr_Earbuds, receives audio from the PC, compresses it, and notifies it back. It has no microphone or speaker path; it stands in for a hearing aid's signal processing and is not itself a hearing aid.

13. Transport over GATT

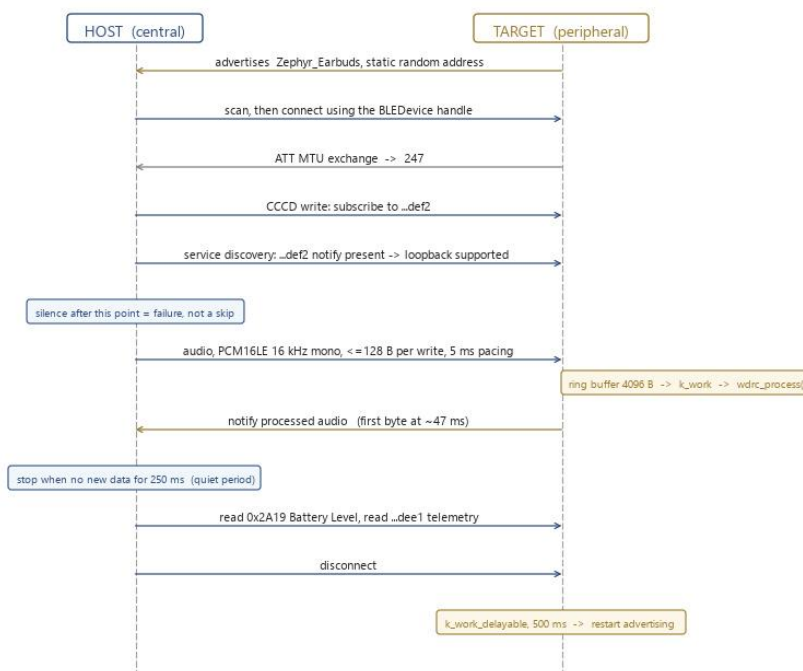


Figure 3 - Bluetooth exchange for a single audio test.

13.1 Device resolution

find_device returns the BLEDevice handle from the scan; the WinRT backend requires the handle rather than an address string, so HA_DEVICE_ADDRESS is resolved through the scanner. Without it the headless pipeline connects to the first advertiser whose name contains "Zephyr". The dashboard does not use that fallback and requires an explicit selection.

13.2 Stream segmentation and termination

Chunk size is $\max(20, \min(128, \text{mtu} - 3))$ rounded down to an even number, so an int16 sample is never split across two writes. Writes use Write Without Response at 5 ms pacing. The receive loop ends when the accumulated length is unchanged for 0.25 s, bounded by 2.5 s.

13.3 Latency and loopback probe

Latency is measured from the first write to the first notification, isolating link and scheduling latency from transfer time. Audio tests first check the GATT table for the processed-audio notify characteristic (...def2). If it is absent, or can neither notify nor indicate, the case skips as a missing feature. If it is present, a burst that returns nothing is a failure: the DSP stopped, a write failed, or the reply timed out. A table that cannot be read raises LoopbackProbeError rather than skipping, because a question that got no answer is not an answer of "no".

13.4 Per-metric degradation

read_metrics emits a key only where the device reported a value and lists the rest under unmeasured. Nothing is substituted; an unmeasured metric scores as risk and its test skips with the reason from risk_engine.UNMEASURED_WHY.

14. Target Firmware

Source, configuration and pre-built images are in src/NRF_Firmware/. The firmware is a Zephyr peripheral exposing three GATT services: an audio service (a write characteristic for audio in, a notify characteristic for processed audio out), a telemetry service (a snapshot characteristic carrying the scored metrics, a build-id characteristic, and a secure characteristic that refuses reads until the link is encrypted, used by the pairing scenario), and the standard Battery Service.

Symbol	Value	Rationale
AUDIO_RING_BYTES	4096	Decouples the ATT write callback from processing
AUDIO_BLOCK_SAMPLES	64	Samples per work-handler iteration
CONFIG_BT_L2CAP_TX_MTU	247	ATT MTU ceiling; the host derives its chunk size from the negotiated value
CONFIG_BT_BUF_ACL_TX_COUNT	10	Default counts stall under streaming load
CONFIG_BT_BUF_EVT_RX_COUNT	12	Zephyr requires RX greater than TX
CONFIG_SYSTEM_WORKQUEUE_STACK_SIZE	4096	Float arithmetic plus the host stack exceed the default
CONFIG_FPU	y	The compressor is single-precision float
CONFIG_BT_MAX_CONN	1	One central at a time
CONFIG_BT_SMP	y	Pairing and bonding for the security scenario

14.1 Deferred notification

The write callback copies into the ring buffer and returns. Compression and bt_gatt_notify run on a work item, because notifying from the ATT callback can block the host stack and drop samples.

14.2 Advertising restart

Advertising resumes from a delayable work item 500 ms after disconnection, with retry. Calling bt_le_adv_start in the disconnected callback runs before teardown completes and leaves the board non-discoverable until reset.

14.3 Identity address

The host finds the board by its advertised name; Windows caches its attribute table by address. After the GATT layout changes, Windows keeps resolving handles from the cached layout and subscription fails with Invalid Handle. Reflashing does not help because the stale data is on the host. The firmware therefore pins a static random

address (HA_STATIC_ADDR), and **an image whose GATT layout differs must advertise under a different address**. The name is unchanged, so the host still finds the board.

15. WDRC Compressor

Hearing loss narrows the range between the quietest audible sound and the loudest comfortable one. A wide dynamic range compressor applies more gain to quiet input and less to loud, fitting everyday sound into that range. The implementation is an envelope follower with separate attack and release coefficients driving a hard-knee gain computer; state persists across calls, so a stream arriving as many small writes is processed continuously.

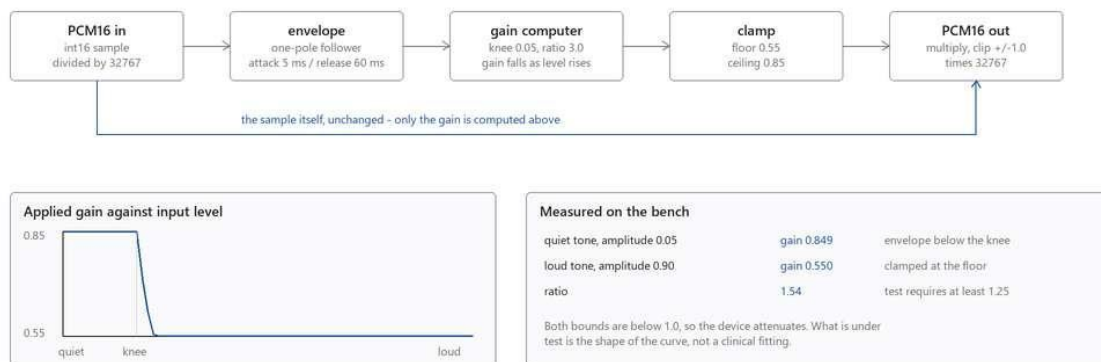


Figure 4 - Signal path, gain curve and measured operating points.

Constant	Value	Definition
WDRC_GAIN_CEIL	0.85	Maximum gain, below the knee
WDRC_GAIN_FLOOR	0.55	Minimum gain; reached at an envelope of about 0.096 and held above it
WDRC_KNEE	0.05	Linear amplitude where compression begins
WDRC_RATIO	3.0	Compression ratio above the knee
WDRC_ATTACK_S	0.005	Attack time constant
WDRC_RELEASE_S	0.060	Release time constant
WDRC_SAMPLE_RATE	16000	Sample rate, Hz

Both gain bounds are below unity, so the residual is proportional to the input and the SNR the host observes is set by the gain alone: $SNR = -20 \log_{10}(1 - g)$, which is 6.94 dB at $g = 0.55$ and 16.48 dB at $g = 0.85$. `test_audio_dsp` asserts $0 < SNR < 50$ dB. The constants and the assertion are coupled; change neither in isolation.

Envelope settling. Recovery from full scale takes about 3250 samples, more than one 2000-sample burst. `test_compression_curve` therefore runs two warm-up bursts, measures the second half of the final burst, and requires a quiet-to-loud gain ratio of at least 1.25; it measures 0.849 against 0.550, a ratio of 1.54. A ratio threshold rather than a bare difference is what separates a working compressor from a fixed-gain stage.

16. Telemetry Wire Format

One characteristic carries a consistent snapshot: `SNAPSHOT_FORMAT "<BBHHHHIIBB"`, 20 bytes, little-endian, version 3.

Field	Type	Decoding
version	uint8	Must equal SUPPORTED_VERSION (3), otherwise TelemetryError

battery_pct	uint8	0xFF decodes to None
current_ma	uint16	0 decodes to None; maps to power
sync_latency_us	uint16	0 or 0xFFFF decode to None; divided by 1000; maps to sync
reconnects	uint16	Link drops since boot; the host subtracts its own disconnects to get retry
stream_errors	uint16	Reported alongside the scored metrics
mem_used	uint32	Work-queue stack high-water, bytes; maps to memory in KiB
mem_free	uint32	Remaining stack, bytes
bond_count	uint8	Number of stored bonds
sec_level	uint8	Link security level; 2 or more means encrypted

Unmeasured values use sentinels (0xFF battery, 0 current, 0xFFFF sync, 0 mem_used and mem_free) and decode to None, never to a number. Version 2 renamed the heap fields to mem_used and mem_free, because the firmware never allocates at run time and heap use would be a constant zero; the version bump makes a host expecting heap bytes refuse the record rather than mislabel it. Version 3 appended bond_count and sec_level (18 -> 20 bytes).

17. Bench Measurements and Results

This is the mapping from requirement wording to the value the adapter takes, with the reading and the limit each requirement states. The readings were taken on the proof-of-concept board from build 355a3760+234c9ec0; they move with the firmware and with the board, so re-read them from a run after any firmware change.

Requirement wording	Measured value	Method	Result (limit)
signal-to-noise	snr_db	Tones at 0.05 and 0.90; signal power over error power	6.9 dB (0-50)
latency	latency_ms	First write to first notification, 2000-sample burst	47 ms (< 2000)
gain for quiet input	quiet_over_loud_gain_percent	RMS gain of the settled half-burst at 0.05 and 0.90	54.5 % (≥ 25)
applied gain shall remain	min_gain, max_gain	Per-sample gain bounds across 0.02-0.99	0.550, 0.850 (0.55-0.85)
opposite sign	loud_sign_flip_percent	Share of samples above half scale returned inverted	0 % (< 1)
attack, release	attack_ms, release_ms	Calibrated DC-step fit	5.004 ms, 59.93 ms (5, 60)
battery level shall be between	battery_percent	Battery Level characteristic 0x2A19 (Battery Service 0x180F)	66 % (0-100)
stack high-water	stack_kib	Telemetry mem_used	0.52 KiB (≤ 3.76)
build identifier	build_id_characters	Build-id characteristic length	17 (≤ 32)

Gain bounds. The firmware truncates toward zero, so a sample's true gain lies between $|out|/|in|$ and $(|out|+1)/|in|$; a violation is reported only when the whole interval is outside the limit.

Compressor timing. The gain curve is first calibrated on the device at twelve constant levels, independent of the firmware's constants. A step from 0.03 to 0.10 and back is converted to an envelope through that curve and fitted; a fit below R-squared 0.995 is rejected. Measured: attack 5.004 ms, release 59.93 ms, R-squared 1.000. WDRC_ATTACK_S sits exactly on the 5 ms REQ-AUD-010 allows, so timing checks carry a 2 % tolerance (canonical.TIMING_TOLERANCE), printed with the verdict.

Verification. tests/test_measurements.py drives every measurement against a float32 port of wdrc_process: it recovers 5 ms and 60 ms within 0.1 %, tracks altered constants, rejects a fixed-gain stage, and fails on a single dropped sample.

In a build id, the part after + is the hash of the sources and build options; the part before it follows the repository revision and changes with every commit. The reasons a case cannot be run on this bench, and what would remove

each, are below. How many fall in each row moves with REQUIREMENTS.md, with the release note's newest section and with the firmware, so read the counts off a run:

Skip reason	Removed by
Boundary or negative case of a measured requirement or release-note limit: the device cannot be driven to exactly the limit, or into the condition	Fault-injection firmware
Supply current: the board cannot measure its own supply	PPK II (HA_POWER_MA, HA_POWER_CSV)
Binaural sync: needs a left and a right device	A second board
Build-system release-note changes, and the placeholder standing for them	Nothing - no device behaviour of their own
Concurrent connections: testing the one-connection limit needs a second central (only when the build's .config supplies CONFIG_BT_MAX_CONN)	A second computer or adapter
Battery temperature: no thermistor on the DK	Hardware
Noise cancellation: no interface for this component on this bench	Firmware that implements ANC

18. Supporting Subsystems of the Adapter

18.1 Audio conversion

The target consumes PCM16LE, 16 kHz, mono. `audio_prep` walks RIFF chunks directly, because the wave module rejects IEEE float and `WAVE_FORMAT_EXTENSIBLE`, and accepts 8/16/24/32-bit integer and 32/64-bit float at any rate and channel count.

18.2 Path resolution

`paths.base_dir` is the source root under a source run and the executable's directory under a frozen build; bundled read-only assets resolve through `sys._MEIPASS`, while logs and generated tests stay writable beside the executable. If that folder is not writable (for example Program Files), output goes to `%LOCALAPPDATA%\AI_Enabled_Regression_Test_Optimization`, and the console says so.

18.3 Optional UART console capture

The host does not use the board's UART; every exchange travels over Bluetooth. To capture the board's console beside each test's log, add `pyserial`, a background reader (`regression/uart_console.py`) and two fixtures gated on `HA_UART_PORT`, and add "serial" and "regression.uart_console" to `hiddenimports` in `build_exe.spec`. On the nRF52840 DK the console is the J-Link CDC UART port at 115200 8N1.

18.4 Loopback failure analysis

The audio loopback did not work until four independent faults were cleared, each masking the next:

Fault	Mechanism
No notify characteristic	The service defined only a write characteristic; nothing could return processed audio.
Advertising not resumed	<code>bt_le_adv_start</code> in the disconnected callback, before teardown, returned an unchecked error.
Stale GATT cache	Windows resolved handles from the pre-change layout (section 14.3).
Event-loop affinity	The fixture subscribed on one <code>asyncio</code> loop and each test created another; bleak delivers notifications to the loop that subscribed. Same loop 400 bytes, new loop 0 bytes.

Console encoding. Non-ASCII characters in `print()` raise `UnicodeEncodeError` on the Windows console code page, so all diagnostic output is ASCII.

Part 3 - Build Procedure

19. Prerequisites

Item	Requirement	Required for
Python	3.12 (CI and the shipped build)	Running from source; building the executable
pip packages	requirements.txt; requirements-llm.txt for the Anthropic engine, used by the planner and by model-written cases (Gemini needs no package)	The host framework
PyInstaller	The shipped build used 6.21	Building the executable only
Operating system (PoC)	Windows 11, 64-bit	The Bluetooth transport is validated on WinRT
Bluetooth LE adapter (PoC)	Bluetooth LE capable	All communication with the board
Target board (PoC)	nRF52840 DK	Running the supplied firmware
nrfjprog or J-Link (PoC)	Vendor supplied	Programming the firmware
nRF Connect SDK v3.4.0 (PoC)	Optional	Rebuilding the firmware from source

20. Host Package

From src/:

```
pip install -r requirements.txt          # numpy, pytest, bleak
pip install -r requirements-llm.txt      # Anthropic engine; Gemini needs none
python -m pytest tests/ -q              # expected: 660 tests collected
```

21. Target Firmware (PoC)

Program the supplied image:

```
nrfjprog --program NRF_Firmware/firmware.hex --chiperase --verify
nrfjprog --reset
# or, from an existing west build directory:
west flash --hex-file NRF_Firmware/firmware.hex
```

Or rebuild it from src/ with the nRF Connect SDK:

```
west build -b nrf52840dk/nrf52840 NRF_Firmware

# require encryption (HA_OPEN_WRITE is cached; use --pristine or a new -d)
west build -b nrf52840dk/nrf52840 --pristine NRF_Firmware -- -DHA_OPEN_WRITE=0
```

Use a separate build directory, or --pristine: HA_OPEN_WRITE is a cached CMake option and persists into later builds. Only the non-default value changes the build id.

Constraint. An image whose GATT attribute layout differs must advertise under a different address (section 14.3). This is the most easily forgotten step when changing firmware.

22. Executable Build

```
cd src
pip install pyinstaller
python -m PyInstaller build_exe.spec --noconfirm --distpath ../bin
```

build_exe.spec produces a onedir bundle, bin/AI_Enabled_Regression_Test_Optimization/. The shipped bin/ holds that folder's contents plus REQUIREMENTS.md, NRF_Firmware/RELEASE_NOTES.md, THIRD_PARTY_NOTICES.txt and User_Guide.pdf. Two properties need explicit declarations:

- **Dynamic plugin discovery.** pytest resolves plugins through entry points, so pytest, _pytest, pluggy, bleak, numpy and the bleak backends are declared in hiddenimports.
- **Absent analysis target.** The generated modules do not exist at build time, so the modules only they import (wave, asyncio, regression.measurements and the canonical pipeline) are declared explicitly and must be kept in step with the templates.

regression/generated_tests/conftest.py is bundled as data: a frozen build has no pytest.ini, so only a conftest.py beside the generated module is loaded. onedir is used rather than onefile so the writable regression/ folder sits beside the executable. The console is disabled. The dashboard re-invokes the executable with --run-pipeline, because a frozen build has no interpreter to launch.

23. Verification and Distribution Readiness

- The host-only suite is the post-build check: it needs no hardware, and every test but three runs without optional dependencies (those three need requirements-llm.txt).
- The executable is unsigned; Smart App Control and similar policies block it until it is signed or exempted.
- Only test material with known terms is distributed. The third-party sample_sound.wav was removed because its terms could not be established; the default input is the project's own test_signal.wav (see CREDITS.md).
- The packaged build bundles the Anthropic client, used by the planner and by model-written cases, and its dependencies; Gemini needs no package. CREDITS.md lists them.

Part 4 - Reference

24. Licensing and Third-Party Components

The project is released under the MIT License; the full text is in LICENSE at the repository root, and every Python source file carries an SPDX-License-Identifier: MIT header, as does each generated test module.

Field	Value
Licence	MIT
Copyright	Copyright (c) 2026 Prajnaana Technologies Pvt. Ltd.
Original author	Dhanya Shree S
Acknowledgement	The idea was mooted by Mahendra Tailor, who has been involved in integrating Bluetooth in hearing aids.

Third-party components retain their own licences and are listed in CREDITS.md.

25. Source File Reference

25.1 Generic core

File	Function
ai_engine/orchestrator.py	Pipeline entry point: derive change, read metrics, score, plan, generate, govern, run, escalate, record.
ai_engine/planner.py	RegressionPlan; the AI planner and the rule planner behind plan_regression.
ai_engine/engines.py	Model-engine interface, registry and selection policy.
ai_engine/llm_client.py	Anthropic client: schema-constrained request, availability check.
ai_engine/gemini_client.py	Gemini client: Generative Language API over urllib; translates PLAN_SCHEMA into the OpenAPI subset Gemini accepts.
ai_engine/generate_tests.py	Writes and ast-checks test ai_generated.py.
ai_engine/adaptive_engine.py	Single-step intensity escalation.
ai_engine/change_spec.py	Release note to typed changes.
ai_engine/test_spec.py	Cases implied by each change type.
ai_engine/requirement_tests.py	Requirement to Verify, boundary and negative cases.
ai_engine/ai_cases.py	Model-proposed requirement cases, inside a closed action and measurement vocabulary (HA_CASE_SOURCE, HA_AI_CASE_LIMIT).
ai_engine/canonical.py	Merges both sources into the canonical document (in memory; --out writes JSON); maps requirements to measurements.
ai_engine/generate_from_canonical.py	Writes test from canonical.py.
change_detection/risk_engine.py	Thresholds, score, slice, reasons for unmeasured metrics.
change_detection/firmware_diff.py	Firmware fingerprints and comparison.
change_detection/firmware_facts.py	Claims in the release note checked against the image.
change_detection/git_changes.py	Source changes since the last tested revision.
governance.py	Acceptance rules and approval, for both generated modules.
kpi.py	The seven KPIs; run, escape and effort records; the history summary for the planner.
catalog.py	Test inventory by category and automation class.
analytics/trend.py	Build-to-build comparison and flaky detection.
reporting.py	JUnit XML to records, logs and summary.
dashboard.py	Tkinter control panel.
paths.py	Source and frozen path resolution.
pytest.ini	pytest configuration for the host-only suite.
build_exe.spec	PyInstaller specification for the packaged application.
requirements.txt, requirements-llm.txt	Host dependencies; requirements-llm.txt adds the Anthropic client, used by the planner and by model-written cases; Gemini needs no package.
regression/artifacts/	KPI history, firmware fingerprints, approvals and baselines/metrics.json (the

threshold-calibration samples).

Paths are relative to src/regression/ unless they start with NRF_Firmware/ or are top-level src/ files.

25.2 Proof-of-concept adapter and assets

File	Function
ble/ble_audio.py	GATT client: scan, connect, stream, loopback probe, battery, telemetry, pairing.
ble/telemetry.py	Telemetry record decoder.
measurements.py	Bench measurements for the requirement suite.
audio_prep.py	Any WAV to PCM16LE 16 kHz mono.
instrumentation/power_analyzer.py	Supply current from a PPK II reading or capture.
ai_engine/scenario_tests.py	Scenario test templates.
conftest.py, generated_tests/conftest.py	Session fixture yielding the connected board.
audio/test_signal.wav	Synthesised fallback stimulus (tools/make_test_signal.py).
REQUIREMENTS.md, NRF_Firmware/RELEASE_NOTES.md	PoC requirements and release note.
NRF_Firmware/src/	Firmware source: main.c, ble_telemetry_service.c/.h, dsp_wdrc.c/.h.
NRF_Firmware/prj.conf, CMakeLists.txt, boards/	Firmware build configuration.
NRF_Firmware/firmware.hex, firmware.bin	Pre-built image, build 355a3760+234c9ec0 (source hash 234c9ec0).
artifacts/firmware/baseline.json	Fingerprint of the shipped image, the first comparison point.

25.3 Tests and tools

tests/ holds the host-only suite in fourteen files: audio_prep, canonical, change_spec, dashboard, firmware_facts, governance, kpi, measurements, paths, pipeline_end_to_end, planner, scenarios, telemetry and test_spec. tools/ holds ble_probe.py and ble_inspect.py (diagnostics), dsp_listen.py (before/after audio), capture_baseline.py (threshold calibration), make_test_signal.py, scenario_matrix.py, scan_secrets.py (the CI secret gate). The figures in docs/ carry counts read from REQUIREMENTS.md and the newest release-note section. docs/generation.png (Figure 2) shows what generates what: the inputs, the two generation paths, both generated modules, the pytest run and the outputs.

26. Project Structure

```
AI-Enabled-Regression-Test-Optimization/  
|-- README.md CREDITS.md SECURITY.md LICENSE  
|-- .github/workflows/regression.yml    host job + self-hosted hardware job  
|-- docs/  
|   |-- architecture.png overview.png pipeline.png generation.png  
|   |-- figure3_window.png             User Guide figure 3  
|   |-- Technical_Document.pdf  
|-- bin/                               packaged Windows application  
|   |-- AI_Enabled_Regression_Test_Optimization.exe  
|   |-- _internal/  
|   |-- REQUIREMENTS.md NRF_Firmware/RELEASE_NOTES.md THIRD_PARTY_NOTICES.txt  
|   |-- User_Guide.pdf  
|-- src/  
|   |-- conftest.py pytest.ini build_exe.spec  
|   |-- requirements.txt requirements-llm.txt REQUIREMENTS.md  
|   |-- NRF_Firmware/                  PoC firmware source, config, images, release note  
|   |-- regression/  
|       |-- ai_engine/                  planner, engines, generators, derivation  
|       |-- change_detection/           risk, firmware and source change analysis  
|       |-- analytics/ instrumentation/ ble/ audio/ generated_tests/ artifacts/  
|       |-- governance.py kpi.py catalog.py reporting.py dashboard.py  
|       |-- measurements.py audio_prep.py paths.py  
|-- tests/                             660 host-only tests  
|-- tools/                             diagnostics and bench helpers
```